

High-Level Design : Centralized Dynamic Routing (Proof of Concept)

Madhav P Nair and Devapriya Pradeep

September 2026

1 Objective of the PoC

Here, we demonstrate a **centralized dynamic router** using **learned dynamic edge costs**.

Simply, if we have n number of vehicles to go from point A to point B and there are m number of routes connecting these two points, we will assign n_1 vehicles to path m_1 , n_2 vehicles to path $p_2, ..$ and n_k vehicles to path m_l such that the sum of the time taken by each vehicle to complete the journey is minimum (**System Optimal**) or every vehicle should reach the destination in a fixed time deadline.

Here, $n_1 + n_2 + .. + n_k = n$ and $p_1 + p_2 + .. + p_l = m$

2 Conceptual Components

2.1 GNN

The junctions/cities are represented by the the vertices of the graph. We treat the weights of the edges connecting two points as estimated travel time rather than static physical distance between the points. This dynamic edge weights can be fetched from a GNN model learned using historical traffic-data sets of the city.

Some potential input features for the model are :

- Date and Time
- Weather
- Road segment length
- Speed limit
- Historical average delay

- Current traffic volume
- in-degree and out-degree

2.2 Graph Solver - Shortest Path Calculation

We find routes for each vehicle so that we can minimize the total time taken by all vehicles or each vehicle reaches the destination within a prescribed time. Different approaches to achieve the same are :

- We consider the PS as an **optimization problem** that minimizes the total travel time of vehicles present. It accounts for the edge weights predicted by the GNN of different paths, along with the congestion caused by a vehicle traveling on a path to assign different vehicles to different paths so that system optimality is attained.

For example, if three paths exist from point A to B and they have edgeweights 10, 20 and 100 (here edgeweights are related to time taken for travel) then the algorithm assigns n_1 vehicles to path 1, n_2 vehicles to path 2 and n_3 vehicles to path 3 such that

$$(10 + x * n_1) * n_1 + (20 + x * n_2) * n_2 + (100 + x * n_3) * n_3$$

is minimized.

This collapses into a **quadratic programming problem**. In general, we try to minimize

$$\sum n_i * (w_i + p * n_i)$$

where n_i represents the number of vehicles passing through path i , w_i is the total weight of path i and p is the fixed penalty due to a vehicle on a path.

- We assign a shortest path to some vehicle v . We have to increase the edge weights of the edges of the path to which the vehicle v has been assigned since it consumes some capacity of the path. We successively find the shortest path for all the vehicles while also considering this extra load on the assigned paths. This extra load in the path is accounted using some **congestion penalty** added to the edge weight predicted by the model.

To implement the above idea, one proposed solution can be :

- Find all or preferably the top k paths from the source to destination.
- Put these in a priority queue having the shortest path as the root node
- Now assign a vehicle v to path p_{top} which is the root node
- Add the fixed congestion penalty x to each edge of p_{top} (This is how we tackle the problem of overlapping edges in different paths)

- We increment the length of all paths having the edges being updated
- Assign next vehicle to the new root node of the queue (Now the root node of the queue is the current shortest path)

Thus we can avoid calling Dijkstra for all n vehicles by caching some relevant routes in memory

Note : Running Dijkstra or any other SSSP algorithm for each vehicle (n times) will be inefficient. It is recommended to output the shortest path for each vehicle using relevant information from the iterations for previous vehicles since the underlying graph structure is unchanged despite of the changing edge weights due to congestion penalty.

The algorithm will be run at regular intervals.

If a penalty is assigned to a path because of a vehicle, it is removed from the path as soon as the vehicle reaches the destination.

3 References

- <https://www.tandfonline.com/doi/full/10.1080/1206212X.2026.2642008>
- <https://researchportal.helsinki.fi/en/publications/an-intelligent-multi-depot-vehicle->
- https://en.wikipedia.org/wiki/Frank%E2%80%93Wolfe_algorithm for solving the quadratic convex combinational problem